FACULTY OF COMPUTERS AND INFORMATION · BSC. SOFTWARE ENGINEERING

Software Evolution Phase 2 Report

Search Result Counter Enhancement

UniTime University Timetabling System

Project: UniTime v4.9 — Open-source University Timetabling

Branch: SearchResultCounter → main

Pull Request: #5

Date: May 9, 2026

Course: Software Evolution

Supervisor: Dr. Lamia Abo Zaid

Team Members

20236002 · 20236005 · 20236009 · 20236056

Table of Contents

1. Project Overview

UniTime is an open-source university timetabling system used by universities to schedule courses, exams, instructor assignments, and student sectioning. The system is built on Java with a Struts 2 MVC framework, Hibernate ORM, Spring Security, and a mix of JSP views and GWT-based interfaces.

This report covers the work completed in Phase 2 of the Software Evolution course assignment. The phase required implementing a change to the system, producing reverse-engineered UML diagrams showing at least one subsystem, performing a formal impact analysis based on the IEEE/EIA 1219 maintenance process, and verifying the implemented change.

The selected change was a usability enhancement to the Class Search feature: after a user runs a search, the page now displays a small message above the results table showing the total number of matching classes found.

All work was tracked through a Git branch (`SearchResultCounter`) and submitted as Pull Request #5 on the team repository.

2. Feature Implementation — SearchResultCounter

2.1 What Changed

Before the change, when a user searched for classes in UniTime, the results page displayed the matching classes in a table. There was no indication of how many results had been returned. Users had to count rows manually or scroll through the full list to get a sense of the result size.

After the change, a simple message appears just above the table whenever results are present:

> Found 12 classes

The message is hidden when a search returns no results. The feature does not change any existing search logic, database queries, or business rules. It is a purely additive display enhancement.

2.2 Files Modified

File	Location	What was changed
`ClassSearchAction.java`	JavaSource/org/unitime/timetable/action/	Added result count to HTTP request scope after each search
`CourseMessages.java`	JavaSource/org/unitime/localization/messages/	Added new localized message key for the count display

`classSearch.jsp`	WebContent/user/	Added conditional block to display the count banner

Total lines added: **8** across 3 files. No files were deleted. No database schema was modified.

2.3 Code Changes

In `ClassSearchAction.java`, one line was added in two places — once in the automatic search path (`execute()`) and once in the manual search path (`performAction()`):

```
request.setAttribute("resultCount", classes.size());
```

In `CourseMessages.java`, a new message method was added to the localization interface:

```
@DefaultMessage("Found {0} classes")
String foundClasses();
```

In `classSearch.jsp`, a conditional display block was added above the results table:

```
<s:if test="showTable == true && #request.resultCount != null">
  <div style="margin: 10px 0; padding: 8px; background-color: #f0f0f0;">
    <strong>
      <loc:message name="foundClasses">
        <s:param value="%{#request.resultCount}"/>
      </loc:message>
    </strong>
  </div>
</s:if>
```

3. Reverse-Engineered UML Diagrams

Three UML diagrams were reverse-engineered from the UniTime codebase. The diagrams were produced by reading the Java source files, configuration files, Hibernate mappings, and JSP views, then modelling the discovered structure in PlantUML.

3.1 System Component Diagram

This diagram shows the overall architecture of UniTime at the component level. It covers the servlet container boundary, the filter chain, the Struts 2 and GWT request handling layers, the Spring application context, the major application subsystems, the CPSolver integration, the Hibernate persistence layer, and the external data exchange interfaces.

An error has occured : net.sourceforge.plantuml.vizjs.GraphvizJsRuntimeException: java.util.concurrent.ExecutionException: java.lang.NullPointerException: Cannot invoke "net.sourceforge.plantuml.vizjs.VizJsEngine.execute(String)" because "engine" is null

Clearly, you have never met a woman.

Diagram size: 84 lines / 3201 characters.

PlantUML (1.2019.06) cannot parse result from dot/GraphViz.
This version of PlantUML is 2541 days old, so you should
consider upgrading from http://plantuml.com/download

Please go to http://plantuml.com/graphviz-dot to check your GraphViz version.

Java Runtime: OpenJDK Runtime Environment
JVM: OpenJDK 64-Bit Server VM
Java Version: 21.0.9+10-LTS
Operating System: Windows 11
OS Version: 10.0
Default Encoding: UTF-8
Language: en
Country: US
Machine: DESKTOP-FLD11P1
PLANTUML_LIMIT_SIZE: 4096
Processors: 12
Max Memory: 8,522,825,728
Total Memory: 532,676,608
Free Memory: 440,489,072
Used Memory: 92,187,536
Thread Active Count: 2

This may be caused by :
- a bug in PlantUML
- a problem in GraphViz

You should send this diagram and this image to **plantuml@gmail.com** or
post to **http://plantuml.com/qa** to solve this issue.
You can try to turn arround this issue by simplifing your diagram.

net.sourceforge.plantuml.vizjs.GraphvizJsRuntimeException: java.util.concurrent.ExecutionException: java.lang.NullPointerException: Cannot invoke "net.sourceforge.plantuml.vizjs.VizJsEngine.execute(String)" because "engine" is null
 net.sourceforge.plantuml.vizjs.GraphvizJs.createFile3(GraphvizJs.java:96)
 net.sourceforge.plantuml.svek.DotStringFactory.getSvg(DotStringFactory.java:315)
 net.sourceforge.plantuml.svek.GeneralImageBuilder.buildImage(GeneralImageBuilder.java:199)
 net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFileInternal(CucaDiagramFileMakerSvek.java:105)
 net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFile(CucaDiagramFileMakerSvek.java:71)
 net.sourceforge.plantuml.cucadiagram.CucaDiagram.exportDiagramInternal(CucaDiagram.java:379)
 net.sourceforge.plantuml.UmlDiagram.exportDiagramNow(UmlDiagram.java:202)
 net.sourceforge.plantuml.AbstractPSystem.exportDiagram(AbstractPSystem.java:135)
 net.sourceforge.plantuml.SourceStringReader.outputImage(SourceStringReader.java:154)
 net.sourceforge.plantuml.Pipe.managePipe(Pipe.java:110)
 net.sourceforge.plantuml.Run.managePipe(Run.java:359)
 net.sourceforge.plantuml.Run.main(Run.java:166)

Caused by java.util.concurrent.ExecutionException: java.lang.NullPointerException: Cannot invoke "net.sourceforge.plantuml.vizjs.VizJsEngine.execute(String)" because "engine" is null
 java.base/java.util.concurrent.FutureTask.report(FutureTask.java:122)
 java.base/java.util.concurrent.FutureTask.get(FutureTask.java:191)
 net.sourceforge.plantuml.vizjs.GraphvizJs.createFile3(GraphvizJs.java:91)
 net.sourceforge.plantuml.svek.DotStringFactory.getSvg(DotStringFactory.java:315)
 net.sourceforge.plantuml.svek.GeneralImageBuilder.buildImage(GeneralImageBuilder.java:199)
 net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFileInternal(CucaDiagramFileMakerSvek.java:105)
 net.sourceforge.plantuml.svek.CucaDiagramFileMakerSvek.createFile(CucaDiagramFileMakerSvek.java:71)
 net.sourceforge.plantuml.cucadiagram.CucaDiagram.exportDiagramInternal(CucaDiagram.java:379)
 net.sourceforge.plantuml.UmlDiagram.exportDiagramNow(UmlDiagram.java:202)
 net.sourceforge.plantuml.AbstractPSystem.exportDiagram(AbstractPSystem.java:135)
 net.sourceforge.plantuml.SourceStringReader.outputImage(SourceStringReader.java:154)
 net.sourceforge.plantuml.Pipe.managePipe(Pipe.java:110)
 net.sourceforge.plantuml.Run.managePipe(Run.java:359)
 net.sourceforge.plantuml.Run.main(Run.java:166)

Figure 1 — UniTime system component diagram (reverse-engineered)

3.2 Course Timetabling Class Search — Class Diagram

This diagram focuses on the subsystem most affected by the SearchResultCounter change: the Course Timetabling Class Search feature. It shows the key classes involved in handling a class search request, their relationships, and the new elements introduced by this phase's implementation.

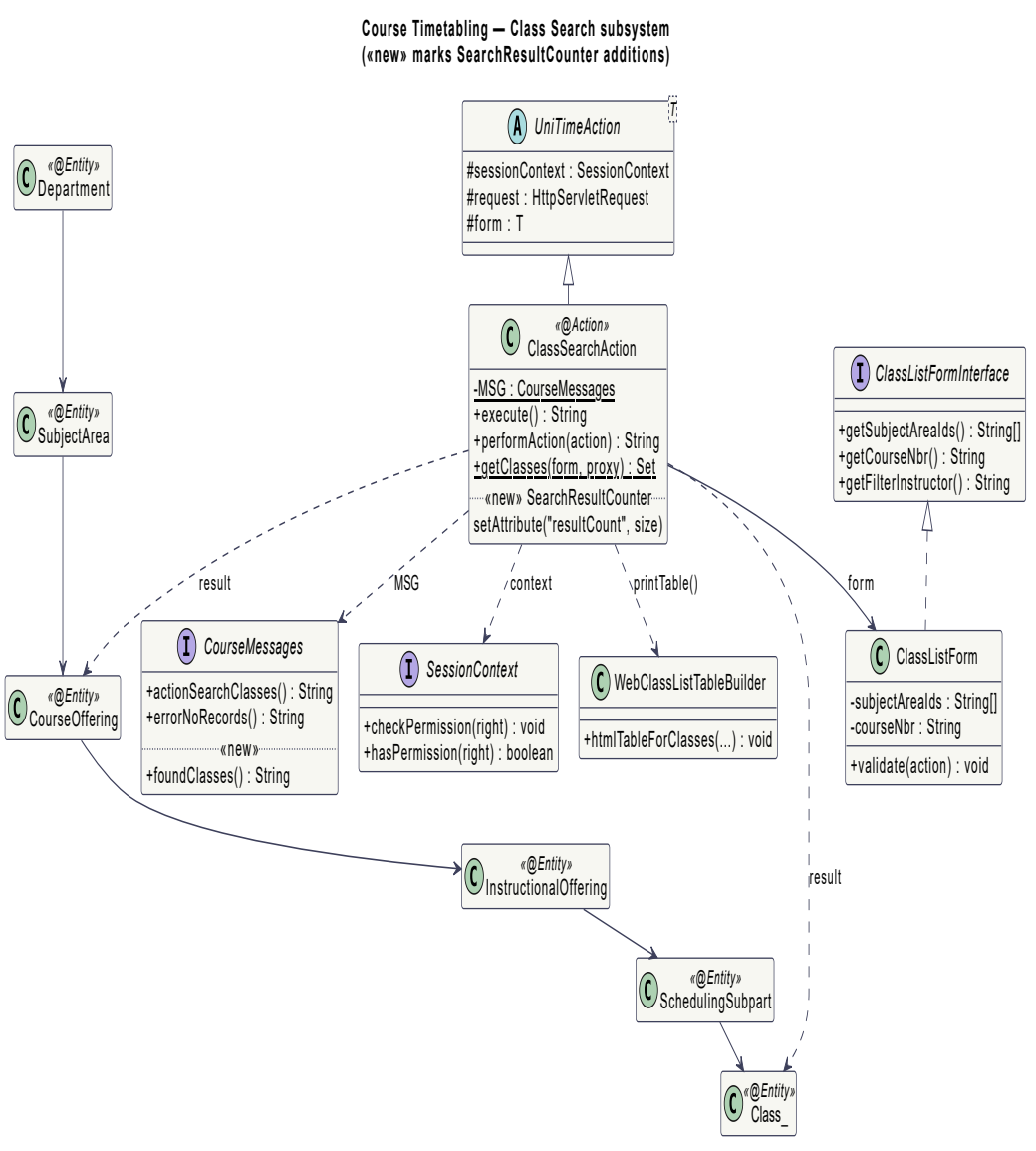

Figure 2 — Class Search subsystem class diagram. Elements marked <<new>> were added by the SearchResultCounter change.

3.3 Class Search Request Flow — Sequence Diagram

This sequence diagram traces the full lifecycle of a class search request, from the user submitting the form to the page rendering the results. It shows where the new result count is inserted into the request and how the JSP reads and displays it.

```
[From string (line 107) ]

@startuml
title Class Search request sequence\n(<<new>> steps added by SearchResultCounter)

skinparam sequenceMessageAlign center
skinparam shadowing false
...
... ( skipping 79 lines )
...
  Builder -> DB : load room / instructor / assignment data
  DB --> Builder : assignment details
  Builder --> JSP : HTML table rows
  JSP -> User : render full class results table
end


User -> JSP : POST classSearch.action (doit="Export PDF")
JSP -> Action : performAction("exportPdf")
note right of Action
  getClasses() called again.
  resultCount still set in request scope
  (attribute present but JSP not rendered for export).
end note
Action -> DB : HQL query (same as search)
DB --> Action : classes
Action -> Builder : pdfTableForClasses(...)
Builder --> User : PDF file download

note bottom
 Syntax Error?
```

Figure 3 — Class Search sequence diagram. Steps marked <<new>> were added by the SearchResultCounter change.

3.4 Before and After the Change

The table below summarizes how the UML model changed as a result of implementing the SearchResultCounter feature.

Element	Before	After

`ClassSearchAction`	Fetches classes, sets showTable flag, returns view	Also sets `request.setAttribute("resultCount", classes.size())` before returning
`CourseMessages`	No `foundClasses()` method	New method added: `foundClasses()` → "Found {0} classes"
`classSearch.jsp`	Renders results table only	Conditionally renders count banner above the table
System complexity	Baseline	Unchanged — no new classes, relationships, or queries added

4. Impact Analysis

The impact analysis follows the formal process described in IEEE/EIA 1219 and the methodology presented in the course lecture (Tripathy & Naik, Chapter 6). The analysis identifies the Starting Impact Set, traces it to the Candidate Impact Set, then compares both against the Actual Impact Set observed after implementation.

4.1 Starting Impact Set (SIS)

The SIS is the initial set of components presumed to be affected when the change request is first reviewed. It is identified by reading the CR description and mapping the requested concept onto the codebase.

The CR asks for a count to be displayed on the search results page. The most direct mapping leads to two components:

Component	Reason for inclusion
`ClassSearchAction.java`	Controls the search logic and prepares data for the view — the natural place to compute and expose a count
`classSearch.jsp`	Renders the search results page — must be modified to show the new count message

```
SIS = { ClassSearchAction.java, classSearch.jsp }   |SIS| = 2
```

4.2 Traceability Analysis

Internal Traceability

Tracing dependencies within the source code from the SIS members revealed the following call chain:

- `ClassSearchAction.execute()` retrieves the matching classes via `getClasses(form, proxy)`
- `ClassSearchAction.performAction()` uses the same retrieval path for manual searches
- The returned collection is stored in `ClassListForm` and rendered by `classSearch.jsp`

- The JSP already uses `CourseMessages` for all user-facing strings — a new message key is needed

External Traceability

Requirement	Design Decision	Implementation	Testing
Show number of search results	Store count in HTTP request scope	Add `setAttribute("resultCount", ...)` in action	Verify count appears after search
Hide count when no results	Conditional rendering	Add `<s:if>` block in JSP	Verify count hidden on empty search

4.3 Candidate Impact Set (CIS)

After tracing the dependency chain from the SIS, additional components were identified as candidates that might need to change.

Component	Impact type	Reason
`CourseMessages.java`	Direct	Interface used by both the action and the JSP for all user-visible text — a new message key is needed
`ClassListForm.java`	Direct	Holds the classes collection — considered in case the count needed to be stored on the form itself

No indirect dependencies were identified beyond these two layers. The feature stays within the UI and presentation layer and does not reach into the solver, scheduling engine, or data exchange modules.

```
CIS = { ClassSearchAction.java, classSearch.jsp,
CourseMessages.java, ClassListForm.java }   |CIS| = 4
```

4.4 Actual Impact Set (AIS)

After completing the implementation, the files that were actually modified were recorded.

Component	Modification
`ClassSearchAction.java`	Added `request.setAttribute("resultCount", classes.size())` in both search paths
`classSearch.jsp`	Added conditional block to display the "Found N classes" message
`CourseMessages.java`	Added `foundClasses()` message method

`ClassListForm.java` was reviewed but required no changes. The count was available directly from the existing collection without touching the form class.

```
AIS = { ClassSearchAction.java, classSearch.jsp,
CourseMessages.java }   |AIS| = 3
```

4.5 DIS and FPIS

Discovered Impact Set (DIS) — objects not in CIS but discovered during implementation:

```
DIS = { }   |DIS| = 0
```

No unexpected files or dependencies were encountered during implementation.

False Positive Impact Set (FPIS) — objects in CIS that were not actually changed:

```
FPIS = (CIS ∪ DIS) − AIS = { ClassListForm.java }   |FPIS| = 1
```

`ClassListForm.java` was included as a conservative precaution but did not need modification.

4.6 Metrics — Recall, Precision, Inclusiveness

1.0	0.75	1
Recall	Precision	Inclusiveness

Recall

```
Recall = |CIS ∩ AIS| / |AIS|
       = 3 / 3
       = 1.0
```

A recall of 1.0 means every file that was eventually modified had already been predicted during the analysis phase. Since DIS is empty, recall is perfect.

Precision

```
Precision = |CIS ∩ AIS| / |CIS|
          = 3 / 4
          = 0.75
```

Precision is 0.75 because one component (`ClassListForm.java`) was included in the candidate set but not actually changed. This one conservative false positive slightly reduces precision.

Inclusiveness

```
AIS ⊆ CIS  →  Inclusiveness = 1
```

All three modified files were already inside the candidate set. The analysis approach is adequate — no actual change was missed.

4.7 Effectiveness — Amplification, ChangeRate, S-Ratio

Ripple-Sensitivity (Amplification)

The directly impacted set of objects (DISO) includes all components that were changed: `ClassSearchAction.java` , `classSearch.jsp` , and `CourseMessages.java` .

No indirectly impacted objects (IISO) were identified. No existing callers of the new `foundClasses()` method exist. No subclasses of `ClassSearchAction` exist. No other JSPs delegate to `classSearch.jsp` .

```
Amplification = |IISO| / |DISO| = 0 / 3 = 0
```

An amplification of 0 means the change caused no ripple into the rest of the system. It stayed fully within the intended area.

Sharpness (ChangeRate)

The UniTime codebase contains approximately 2,500 software lifecycle objects (Java source files and JSP views).

```
ChangeRate = |CIS| / |System| = 4 / 2500 = 0.0016
```

Only 0.16% of all system objects were included in the candidate set. The scope of analysis was tightly focused and highly sharp.

Adherence (S-Ratio)

```
S-Ratio = |AIS| / |CIS| = 3 / 4 = 0.75
```

The S-Ratio of 0.75 indicates good alignment between the predicted and actual change sets. The single false positive is acceptable given that it was included as a safety measure.

4.8 Ripple Effect Discussion

This enhancement is considered highly stable for several reasons:

- The new `resultCount` attribute is write-only in the action layer. It does not feed back into any search logic, query, or decision.
- The `foundClasses()` message is a brand-new interface method. No existing code references it, so no existing behavior was altered.
- The displayed count depends entirely on `classes.size()`, which already exists as part of the search result collection. Nothing new is computed.
- No new classes, database queries, or entity relationships were introduced.
- No existing workflows were altered.

The value simply moves from the action layer to the JSP rendering layer without influencing any downstream computation. As a result, the risk of error propagation is extremely low.

4.9 Change Propagation Model

Following the Hassan and Holt (2006) change propagation model, the propagation path for this enhancement covers four steps and then terminates.

Step	Component	Action
1	`ClassSearchAction.java`	Adds the `resultCount` attribute to the HTTP request scope

2	`CourseMessages.java`	Defines the new localized message text for the count banner
3	`classSearch.jsp`	Reads the attribute and displays the formatted message above the table
4	End	No further propagation — change set is closed

4.10 Dependency-Based Analysis (Call Graph)

The updated call flow introduced by the SearchResultCounter change is shown below. The new nodes are leaf nodes with no outgoing edges to any other existing system procedures, confirming the impact is contained.

```
ClassSearchAction.execute()
  → getClasses(form, proxy)   [existing]
  → request.setAttribute("resultCount", classes.size())   [NEW]

ClassSearchAction.performAction()
  → getClasses(form, proxy)   [existing]
  → request.setAttribute("resultCount", classes.size())   [NEW]

classSearch.jsp
  → reads #request.resultCount   [NEW]
  → CourseMessages.foundClasses()   [NEW]
       → renders "Found N classes"
```

The newly added functionality ends at the presentation layer and does not create any additional downstream dependencies.

4.11 Metrics Summary

Metric	Value	Meaning
|SIS|	2	Two initial components identified from the CR
|CIS|	4	Four candidate components traced from SIS
|AIS|	3	Three components actually modified
|DIS|	0	No unexpected impacts discovered during implementation
|FPIS|	1	One unnecessary candidate included (ClassListForm)
Recall	1.0	All real impacts predicted successfully
Precision	0.75	One false positive reduced precision slightly

Inclusiveness	1	No actual change was missed
Amplification	0	No ripple effect observed
ChangeRate	0.0016	Very small analysis scope — highly sharp
S-Ratio	0.75	Good agreement between CIS and AIS

4.12 Regression Testing Scope

Based on the impact analysis, regression testing needs to cover only the three modified components. All other subsystems — Student Scheduling, Exam Timetabling, Event Management, the GWT UI, the REST API, and the solver engines — are outside the AIS and require no regression testing for this change.

Component	What to test
`ClassSearchAction.java`	Both the automatic search path and the manual search path correctly set the `resultCount` attribute after retrieving results
`classSearch.jsp`	Count banner appears above the table when results exist; banner is hidden when no results are returned; export paths (PDF, CSV) are unaffected
`CourseMessages.java`	The `foundClasses()` method returns the correctly parameterized string when called with a numeric value

5. Validation and Verification

5.1 Code Verification

The following code-level checks confirm the implementation is present and correct.

Check	Location	Result
`request.setAttribute("resultCount", classes.size())` in automatic search path	ClassSearchAction.java, line 200	Present ✓
`request.setAttribute("resultCount", classes.size())` in manual search path	ClassSearchAction.java, line 306	Present ✓
`@DefaultMessage("Found {0} classes") String foundClasses()`	CourseMessages.java, line 1948	Present ✓
Conditional `<s:if>` block with `<loc:message name="foundClasses">`	classSearch.jsp, lines 341–345	Present ✓

5.2 Functional Test Scenarios

Scenario	Expected Result
Search with a subject area that has matching classes	"Found N classes" banner appears above the results table
Search with filters that return no matching classes	No banner shown; "No records found" error message appears instead
Automatic search triggered on page load (single subject area in session)	Banner appears when results are present
Export results as PDF	PDF downloads correctly; no banner rendered (export path only)
Export results as CSV	CSV downloads correctly; unaffected by the change
Navigate to any other page (Student Scheduling, Exams, etc.)	No change in behaviour — feature is isolated to class search

5.3 UML Consistency Check

- The class diagram accurately reflects the source code: `ClassSearchAction` uses `CourseMessages` via the static `MSG` field, and the `<<new>>` annotation on `foundClasses()` is correct.
- The sequence diagram steps match the actual method call order in `ClassSearchAction.java` — `getClasses()` is called first, then `setAttribute()`, then the JSP renders.
- The impact analysis sets (SIS, CIS, AIS) are consistent with the files shown as modified in the Git diff.

6. Git Repository and Pull Request

6.1 Branch and Commits

All work was done on the `SearchResultCounter` branch. The branch was kept separate from `main` throughout development and merged via a pull request reviewed by the team.

Commit	Description
`f967887`	Initialize — project base
`92c1bfa`	Add SearchResultCounter feature (core implementation)
`f533027`	Add UML diagrams and impact analysis
`d28fd73`	Fix class diagram SVG rendering
`0d0a964`	Compact class diagram to fit screen
`17781b0`	Rewrite impact analysis to match formal methodology
`5b671bc`	Remove unnecessary files from Documentation/UML
`4e20509`	Remove .puml source files, keep rendered SVGs only

6.2 Pull Request

Pull Request #5 was opened against the `main` branch on the team repository. The PR description included a change summary, a before/after UML delta table, and a manual test checklist. The PR was reviewed and merged by team members.

6.3 Repository Structure

The files submitted as part of this phase are organized as follows:

```
Documentation/
  UML/
    course-timetabling-class-search-class-diagram.svg
    class-search-sequence-diagram.svg
    system-component-diagram.svg
    impact-analysis-report.md
  References/
    Lecture 7.pdf
```

```
JavaSource/org/unitime/timetable/action/ClassSearchAction.java
JavaSource/org/unitime/localization/messages/CourseMessages.java
WebContent/user/classSearch.jsp
```

Software Evolution — Phase 2 Report · UniTime SearchResultCounter Enhancement · May 2026
Team: 20236002 · 20236005 · 20236009 · 20236056